41039 Programming 1

Revision of Week 10: Writing files and reliable reports



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 6 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. A writer can target memory

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
StringWriter buffer=new StringWriter();
try(PrintWriter writer=new PrintWriter(buffer)) {
    writer.println("name,mark");
    writer.println("Linh,85");
}
System.out.print(buffer.toString());
```

Your answer

What is the complete output? Which line or rule decides it?

1. A writer can target memory

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
StringWriter buffer=new StringWriter();
try(PrintWriter writer=new PrintWriter(buffer)) {
    writer.println("name,mark");
    writer.println("Linh,85");
}
System.out.print(buffer.toString());
```

Answer

```
name,mark
Linh,85
```

The writer emits the header then the record. The buffer lets the page show exactly what would be written.

2. Python write does not add a newline

Python 3; commit to an outcome before the reveal.

```
from io import StringIO
buffer=StringIO()
buffer.write("tea")
buffer.write("coffee")
print(repr(buffer.getvalue()))
buffer.close()
```

Your answer

What is the complete output? Which line or rule decides it?

2. Python write does not add a newline

Python 3; commit to an outcome before the reveal.

```
from io import StringIO
buffer=StringIO()
buffer.write("tea")
buffer.write("coffee")
print(repr(buffer.getvalue()))
buffer.close()
```

Answer

```
'teacoffee'
```

The two writes concatenate; print adds a newline only when displaying the representation.

3. Java file output and read-back on Ed

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
File file=new File("report.txt");
try(PrintWriter out=new PrintWriter(file,"UTF-8")) {
    out.println("name,mark");
    out.println("Linh,85");
    if(out.checkError()) throw new IOException("write
     failed");
}
try(BufferedReader in=new BufferedReader(new
     InputStreamReader(new FileInputStream(file),"UTF-8"
     ))) {
    String line;
    while((line=in.readLine())!=null) System.out.println
     (line);
}
```

Your answer

Predict the output. Why close before reading back? Do PrintWriter write failures always throw IOException?

3. Java file output and read-back on Ed

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
File file=new File("report.txt");
try(PrintWriter out=new PrintWriter(file,"UTF-8")) {
    out.println("name,mark");
    out.println("Linh,85");
    if(out.checkError()) throw new IOException("write
     failed");
}
try(BufferedReader in=new BufferedReader(new
     InputStreamReader(new FileInputStream(file),"UTF-8"
     ))) {
    String line;
    while((line=in.readLine())!=null) System.out.println
     (line);
}
```

Answer

```
name,mark
Linh,85
```

Closing completes the output before a reader opens the file. Records appear in their written order. PrintWriter stores many write failures instead of throwing them; checkError reports those failures. Closing manages resource lifetime but does not prove that every write succeeded.

4. Use CSV tooling when fields need quoting

Python 3; commit to an outcome before the reveal.

```
import csv
from io import StringIO
buffer=StringIO(newline="")
writer=csv.writer(buffer,lineterminator="\n")
writer.writerow(["name","mark"])
writer.writerow(["An, Mai",85])
print(buffer.getvalue(),end="")
buffer.seek(0)
rows=list(csv.reader(buffer))
print(rows[1][0])
buffer.close()
```

Your answer

What is the complete output? Which line or rule decides it?

4. Use CSV tooling when fields need quoting

Python 3; commit to an outcome before the reveal.

```
import csv
from io import StringIO
buffer=StringIO(newline="")
writer=csv.writer(buffer,lineterminator="\n")
writer.writerow(["name","mark"])
writer.writerow(["An, Mai",85])
print(buffer.getvalue(),end="")
buffer.seek(0)
rows=list(csv.reader(buffer))
print(rows[1][0])
buffer.close()
```

Answer

```
name,mark
"An, Mai",85
An, Mai
```

CSV quotes the name containing a comma; the reader returns the original field without the quoting characters.

5. Format records separately from saving them

Java 8; commit to an outcome before the reveal.

```
public class Demo {
    static String format(String name,int mark) { return
     name+","+mark; }
    public static void main(String[] args) {
        String[] names={"Linh","An"};
        int[] marks={85,49};
        for(int i=0;i<names.length;i++) System.out.
     println(format(names[i],marks[i]));
    }
}
```

Your answer

What is the complete output? Which line or rule decides it?

5. Format records separately from saving them

Java 8; commit to an outcome before the reveal.

```
public class Demo {
    static String format(String name,int mark) { return
     name+","+mark; }
    public static void main(String[] args) {
        String[] names={"Linh","An"};
        int[] marks={85,49};
        for(int i=0;i<names.length;i++) System.out.
     println(format(names[i],marks[i]));
    }
}
```

Answer

```
Linh,85
An,49
```

The formatter pairs each name with the corresponding mark; persistence is a separate choice.

6. Round-trip a small document

Python 3; commit to an outcome before the reveal.

```
import json
record={"name":"Linh","marks":[85,90]}
text=json.dumps(record,sort_keys=True)
restored=json.loads(text)
print(restored["name"])
print(sum(restored["marks"]))
print(restored==record)
```

Your answer

What is the complete output? Which line or rule decides it?

6. Round-trip a small document

Python 3; commit to an outcome before the reveal.

```
import json
record={"name":"Linh","marks":[85,90]}
text=json.dumps(record,sort_keys=True)
restored=json.loads(text)
print(restored["name"])
print(sum(restored["marks"]))
print(restored==record)
```

Answer

```
Linh
175
True
```

The round trip preserves the two marks and name; their sum is 175.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: One report, several responsibilities

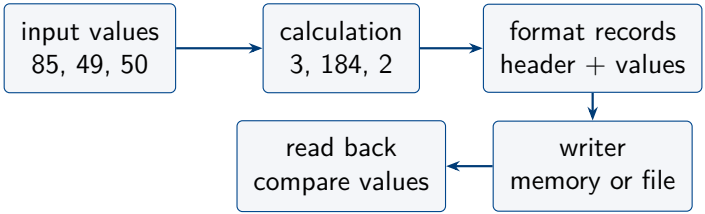
```
import java.io.*;
int[] marks={85,49,50};
int total=0,passed=0;
for(int mark:marks) { total+=mark; if(mark>=50) passed
    ++; }
StringWriter text=new StringWriter();
try(PrintWriter out=new PrintWriter(text)) {
    out.println("count,total,passed");
    out.println(marks.length+","+total+","+passed);
}
System.out.print(text.toString());
```

Course website: Week 10. Complete programs accompany this deck.

Four minutes

Draw the flow and responsibilities after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Draw it: one answer



Output

```
count,total,passed
3,184,2
```

Calculation produces values, formatting chooses a representation, and writing chooses a destination. A read-back test must inspect values and delimiters, not only whether an exception occurred.

Course website: Week 10. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

Emit exactly three CSV lines: a header, Linh,85 and An,49. Names contain no commas or newlines.

```java
import java.io.*;
public class Broken {
    public static void main(String[] args) {
        StringWriter buffer=new StringWriter();
        PrintWriter out=new PrintWriter(buffer);
        String[] names={"Linh","An"};
        int[] marks={85,49};
        out.print("name,mark");
        for(int i=0;i<names.length;i++) {
            out.print(names[i]+marks[i]);
        }
        System.out.print(buffer.toString());
    }
}
```

Spot the errors 1: answers

The repairs

- **L8:** Terminate the header with println.
- **L10:** Insert the field delimiter and terminate every record.
- **L12:** Close the writer before reading back the stored text. A memory writer may hide missing-flush bugs.

Course website: Week 10. Complete programs accompany this deck.

✗ Original program

```
name,markLinh85An49
```

✓ After the repairs

```
name,mark
Linh,85
An,49
```

Spot the errors 2: Python (5 min)

Contract

Append a complete new record to the existing old record. The resulting file must contain old and new on separate lines.

```python
from pathlib import Path
from tempfile import TemporaryDirectory
with TemporaryDirectory() as folder:
    path=Path(folder)/"report.txt"
    path.write_text("old\n",encoding="utf
     -8")
    with path.open("w",encoding="utf-8")
     as out:
        out.write("new")
    with path.open("r",encoding="utf-8")
     as source:
        print(source.read(),end="")
```

Spot the errors 2: answers

The repairs

- **L6:** Use append mode to retain old content.
- **L7:** Add the record terminator explicitly. write does not add a newline.

Course website: Week 10. Complete programs accompany this deck.

✗ Original program

```
new
```

✓ After the repairs

```
old
new
```

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: A writer preserves exact characters

```
import java.io.*;
StringWriter buffer=new StringWriter();
try(PrintWriter out=new PrintWriter(buffer)) {
    out.print("A");
    out.println("B");
    out.println("C");
}
System.out.print(buffer.toString());
System.out.println("D");
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 10. Complete programs accompany this deck.

Predict 1: answer

```
AB
C
D
```

Why

print writes no terminator. The first println completes AB; the next completes C. D goes to standard output after the stored text.

Course website: Week 10. Complete programs accompany this deck.

Predict 2: Quoting and a read-back test

```
import csv
from io import StringIO
text=StringIO(newline="")
writer=csv.writer(text,lineterminator="\n")
writer.writerow(["An, Mai",50])
writer.writerow(["Tea",49])
text.seek(0)
rows=list(csv.reader(text))
print(len(rows))
print(rows[0][0],int(rows[0][1])>=50)
print(rows[1])
```

Course website: Week 10. Complete programs accompany this deck.

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Predict 2: answer

```
2
An, Mai True
['Tea', '49']
```

Why

The CSV writer quotes the comma in the name. The reader reconstructs one field containing that comma; the mark is converted before comparing it numerically.

Course website: Week 10. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

A report with a stable schema

Read n then n marks in 0..100. Print count,total,passed and one CSV record with the number of marks, their sum, and the number at least 50. Standard output is the test destination; the same formatter could feed a file writer.

Example input

```
3
85 49 50
```

Expected output

```
count,total,passed
3,184,2
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution

```java
import java.util.Scanner;
Scanner sc=new Scanner(System.in);
int n=sc.nextInt(),total=0,passed=0;
for(int i=0;i<n;i++) { int mark=sc.nextInt(); total+=mark; if(mark>=50) passed++; }
System.out.println("count,total,passed");
System.out.println(n+","+total+","+passed);
```

Course website: Week 10. Complete programs accompany this deck.

Code it: Python solution

```python
n=int(input())
marks=list(map(int,input().split()))[:n]
print("count,total,passed")
print(f"{n},{sum(marks)},{sum(mark>=50 for mark in marks)}")
```

Course website: Week 10. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`3 / 85 49 50`
Output: `count,total,passed / 3,184,2`

Test 2
`0 / (blank)`
Output: `count,total,passed / 0,0,0`

Test 3
`3 / 0 0 0`
Output: `count,total,passed / 3,0,0`

Test 4
`2 / 100 50`
Output: `count,total,passed / 2,150,2`

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- Specify delimiters, newlines, field order and text encoding.
- Distinguish write, append and read modes.
- Separate computation from formatting and persistence.
- Use quoting for general CSV and inspect values after reading back.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.
Course website: Week 10. Complete programs accompany this deck.